

ARRAYS

7.1 INTRODUCTION

An *array* is a group of related data items that share a common name. For instance, we can define an array name **salary** to represent a *set of salaries* of a group of employees. A particular value is indicated by writing a number called *index number* or *subscript* in brackets after the array name. For example,

salary[10]

represents the salary of the 10th employee. While the complete set of values is referred to as an array, the individual values are called *elements*. Arrays can be of any variable type.

The ability to use a single name to represent a collection of items and to refer to an item by specifying the item number enables us to develop concise and efficient programs. For example, a loop with the subscript as the control variable can be used to read the entire array, perform calculations and, print out the results. In this chapter, we shall discuss how arrays can be defined and used in C.

7.2 ONE-DIMENSIONAL ARRAYS

A list of items can be given one variable name using only one subscript and such a variable is called a *single-subscripted variable* or a *one-dimensional array*. In mathematics, we often deal with variables that are single-subscripted. For instance, we use the equation.

$$A = \frac{\sum_{i=1}^n x_i}{n}$$

to calculate the average of n values of x . The subscripted variable x_i refers to the i th element of x . In C, single-subscripted variable x_i can be expressed as

$x[1], x[2], x[3], \dots, x[n]$

The subscript can begin with number 0. That is

x[0]

is allowed. For example, if we want to represent a set of five numbers, say (35,40,20,57,19), by an array variable **number**, then we may declare the variable **number** as follows

int number[5];

and the computer reserves five storage locations as shown below:

	number[0]
	number[1]
	number[2]
	number[3]
	number[4]

The values to the array elements can be assigned as follows:

```
number[0] = 35;
number[1] = 40;
number[2] = 20;
number[3] = 57;
number[4] = 19;
```

This would cause the array **number** to store the values as shown below:

number [0]	35
number [1]	40
number [2]	20
number [3]	57
number [4]	19

These elements may be used in programs just like any other C variable. For example, the following are valid statements:

```
a = number[0] + 10;
number[4] = number[0] + number [2];
number[2] = x[5] + y[10];
value[6] = number[i] * 3;
```

The subscript of an array can be integer constants, integer variables like *i*, or expressions that yield integers. *C performs no bounds checking and, therefore, care should be exercised to ensure that the array indices are within the declared limits.*

Declaration of Arrays

Like any other variable, arrays must be declared before they are used. The general form of array declaration is

type variable-name[size];

The *type* specifies the type of element that will be contained in the array, such as **int**, **float**, or **char** and the *size* indicates the maximum number of elements that can be stored inside the array. For example,

float height[50];

declares the **height** to be an array containing 50 real elements. Any subscripts 0 to 49 are valid. Similarly,

int group[10];

declares the **group** as an array to contain a maximum of 10 integer constants. Remember, any reference to the arrays outside the declared limits would not necessarily cause an error. Rather, it might result in unpredictable program results.

The C language treats character strings simply as arrays of characters. The *size* in a character string represents the maximum number of characters that the string can hold. For instance,

char name[10];

declares the **name** as a character array (string) variable that can hold a maximum of 10 characters. Suppose we read the following string constant into the string variable **name**.

"WELL DONE"

Each character of the string is treated as an element of the array **name** and is stored in the memory as follows:

'W'
'E'
'L'
'L'
' '
'D'
'O'
'N'
'E'
'\0'

When the compiler sees a character string, it terminates it with an additional null character. Thus, the element **name[9]** holds the null character '\0' at the end. When declaring character arrays, we must always allow one extra element space for the null terminator.

Example 7.1

Write a program using a single-subscripted variable to evaluate the following expressions:

$$\text{Total} = \sum_{i=1}^{10} x_i^2$$

The values of $x_1, x_2, \dots$ are read from the terminal.

Program in Fig. 7.1 uses a one-dimensional array x to read the values and compute the sum of their squares.

Initialization of Arrays

We can initialize the elements of arrays in the same way as the ordinary variables when they are declared. The general form of initialization of arrays is:

static *type* *array-name*[size] = { list of values };

The values in the list are separated by commas. For example, the statement

static int number[3] = { 0,0,0 };

will declare the variable **number** as an array of size 3 and will assign zero to each element. If the number of values in the list is less than the number of elements, then only that many elements will be initialized. The remaining elements will be set to zero automatically. For instance,

static float total[5] = {0.0,15.75,-10};

will initialize the first three elements to 0.0, 15.75, and -10.0 and the remaining two elements to zero.

Note that we have used the word 'static' before *type* declaration. This declares the variable as a *static variable*. More about static and other class of variables are discussed in Chapter 9.

Note: Under the old version (the K & R standard), automatic arrays cannot be initialized. Only external and static arrays may be initialized. However, the ANSI standard permits arrays with **auto** storage class to be initialized. That is, the keyword **static** could be omitted, when an ANSI compiler is used. Since ANSI compilers accept both the versions, we use the storage class **static** in our programs so that they can run under both the K & R and ANSI compilers.

The *size* may be omitted. In such cases, the compiler allocates enough space for all initialized elements. For example, the statement

static int counter[] = {1,1,1,1};

will declare the **counter** array to contain four elements with initial values 1. This approach works fine as long as we initialize every element in the array.

Character arrays may be initialized in a similar manner. Thus, the statement

static char name[] = {'J','O','h','n'};

declares the **name** to be an array of four characters, initialized with the string "John".

Initialization of arrays in C suffers two drawbacks.

1. There is no convenient way to initialize only selected elements.
2. There is no shortcut method for initializing a large number of array elements like the one available in FORTRAN.

Program

```

/*****
PROGRAM SHOWING ONE-DIMENSIONAL ARRAY
*****/
main( )
{
    int i;
    float x[10], value, total ;

    /*.....READING VALUES INTO ARRAY ..... */
    printf("ENTER 10 REAL NUMBERS\n");
    for( i = 0; i < 10 ; i++ )
    {
        scanf("%f", &value);
        x[i] = value ;
    }

    /*.....COMPUTATION OF TOTAL..... */
    total = 0.0 ;
    for( i = 0 ; i < 10 ; i++ )
        total = total + x[i] * x[i] ;

    /*.....PRINTING OF x[i] VALUES AND TOTAL ..... */
    printf("\n");
    for( i = 0 ; i < 10 ; i++ )
        printf("x[%2d] = %5.2f\n", i+1, x[i]) ;
    printf("\ntotal = %.2f\n", total) ;
}

```

Output

```

ENTER 10 REAL NUMBERS
1.1 2.2 3.3 4.4 5.5 6.6 7.7 8.8 9.9 10.10
x[ 1] = 1.10
x[ 2] = 2.20
x[ 3] = 3.30
x[ 4] = 4.40
x[ 5] = 5.50
x[ 6] = 6.60
x[ 7] = 7.70
x[ 8] = 8.80
x[ 9] = 9.90
x[10] = 10.10
Total = 446.86

```

Fig. 7.1 Program to illustrate one-dimensional array

Consider an array of size, say 100. All the 100 elements have to be explicitly initialized. There is no way to specify a repeat count. In such situations, it would be better to use a for loop to initialize the elements. Consider the following segment of a C program:

```

-----
for (i = 0; i < 100; i = i+1)
{
    if i < 50
        sum[i] = 0.0;
    else
        sum[i] = 1.0;
}
-----

```

The first 50 elements of the array `sum` are initialized to zero while the remaining 50 elements are initialized to 1.0.

Example 7.2

Given below is the list of marks obtained by a class of 50 students in an annual examination.

```

43 65 51 27 79 11 56 61 82 09
25 36 07 49 55 63 74 81 49 37
40 49 16 75 87 91 33 24 58 78
65 56 76 67 45 54 36 63 12 21
73 49 51 19 39 49 68 93 85 59

```

Write a program to count the number of students belonging to each of the following groups of marks: 0-9, 10-19, 20-29,.....,100.

The program coded in Fig. 7.2 uses the array `group` containing 11 elements, one for each range of marks. Each element counts those values falling within the range of values it represents.

For any value, we can determine the correct group element by dividing the value by 10. For example, consider the value 59. The integer division of 59 by 10 yields 5. This is the element into which 59 is counted.

Note that the numbers with fractional part are rounded off to the nearest integer before the integer division occurs.

7.3 TWO-DIMENSIONAL ARRAYS

So far we have discussed the array variables that can store a list of values. There will be situations where a table of values will have to be stored. Consider the following data table, which shows the value of sales of three items by four salesgirls:

	Item1	Item2	Item3
Salesgirl #1	310	275	365
Salesgirl #2	210	190	325
Salesgirl #3	405	235	240
Salesgirl #4	260	300	380

The table contains a total of 12 values, three in each line. We can think of this table as a matrix consisting of four *rows* and three *columns*. Each row represents the values of sales by a particular salesgirl and each column represents the values of sales of a particular item.

Program

```

/*.....PROGRAM FOR FREQUENCY COUNTING.....*/
#define MAXVAL 50
#define COUNTER 11
main( )
{
    float    value[MAXVAL];
    int      i, low, high;
    static int group[COUNTER] = { 0,0,0,0,0,0,0,0,0,0,0};

    /*.....READING AND COUNTING.....*/
    for(i = 0; i < MAXVAL; i++)
    {
        /*...READING OF VALUES...*/
        scanf("%f", &value[i]);
        /*...COUNTING FREQUENCY OF GROUPS...*/
        ++ group[ (int) (value[i] + 0.5) / 10] ;
    }

    /*.....PRINTING OF FREQUENCY TABLE.....*/
    printf("\n");
    printf("GROUP    RANGE    FREQUENCY\n\n");
    for( i = 0 ; i < COUNTER ; i++ )
    {
        low = i * 10 ;
        if(i == 10)
            high = 100 ;
        else
            high = low + 9 ;
        printf("   %2d    %3d to %3d    %d\n",
               i+1, low, high, group[i] ) ;
    }
}

```

Output

```

43 65 51 27 79 11 56 61 82 09
25 36 07 49 55 63 74 81 49 37
40 49 16 75 87 91 33 24 58 78
65 56 76 67 45 54 36 63 12 21
73 49 51 19 39 49 68 93 85 59

```

(input data)

GROUP	RANGE	FREQUENCY
1	0 to 9	2
2	10 to 19	4
3	20 to 29	4
4	30 to 39	5

5	40 to 49	8
6	50 to 59	8
7	60 to 69	7
8	70 to 79	6
9	80 to 89	4
10	90 to 99	2
11	100 to 100	0

Fig. 7.2 Program for frequency counting

In mathematics, we represent a particular value in a matrix by using two subscripts such as v_{ij} . Here v denotes the entire matrix and v_{ij} refers to the value in the i th row and j th column. For example, in the above table v_{23} refers to the value 325.

C allows us to define such tables of items by using *two-dimensional* arrays. The table discussed above can be defined in C as

v[4][3]

Two-dimensional arrays are declared as follows: -

```
type array_name [row_size][column_size];
```

Note that unlike most other languages, which use one pair of parentheses with commas to separate array sizes, C places each size in its own set of brackets.

Two dimensional arrays are stored in memory as shown in Fig. 7.3. As with the single-dimensional arrays, each dimension of the array is indexed from zero to its maximum size minus one; the first index selects the row and the second index selects the column within that row.

	Column0	Column1	Column2
	↓ [0][0]	↓ [0][1]	↓ [0][2]
Row 0 →	310	275	365
	[1][0]	[1][1]	[1][2]
Row 1 →	210	190	325
	[2][0]	[2][1]	[2][2]
Row 2 →	405	235	240
	[3][0]	[3][1]	[3][2]
Row 3 →	260	300	380

Fig. 7.3 Representation of a two-dimensional array in memory

Example 7.3

Write a program using a two-dimensional array to compute and print the following information from the table of data discussed above:

- Total value of sales by each girl.
- Total value of each item sold.
- Grand total of sales of all items by all girls.

The program and its output are shown in Fig. 7.4. The program uses the variable **value** in two-dimensions with the index **i** representing girls and **j** representing items. The following equations are used in computing the results:

$$(a) \text{ Total sales by } m\text{th girl} = \sum_{j=0}^2 \text{value}[m][j] \\ (\text{girl_total}[m])$$

$$(b) \text{ Total value of } n\text{th item} = \sum_{i=0}^3 \text{value}[i][n] \\ (\text{item_total}[n])$$

$$(c) \text{ Grand total} = \sum_{i=0}^3 \sum_{j=0}^2 \text{value}[i][j] \\ = \sum_{i=0}^3 \text{girl_total}[i] \\ = \sum_{j=0}^2 \text{item_total}[j]$$

Program

```

/*****
/*      PROGRAM SHOWING TWO-DIMENSIONAL ARRAYS      */
*****/

#define    MAXGIRLS    4
#define    MAXITEMS    3

main( )
{
    int    value[MAXGIRLS][MAXITEMS];
    int    girl_total[MAXGIRLS], item_total [MAXITEMS];
    int    i, j, grand_total;

    /*.....READING OF VALUES AND COMPUTING girl_total..... */

    printf("Input data\n");
    printf("Enter values, one at a time, row-wise\n\n");
    for( i = 0 ; i < MAXGIRLS ; i++ )
    {

```

```

        girl_total[i] = 0;
        for( j = 0 ; j < MAXITEMS ; j++ )
        {
            scanf("%d", &value[i][j]);
            girl_total[i] = girl_total[i] + value[i][j];
        }
    }

    /* .....COMPUTING item_total ..... */
    for( j = 0 ; j < MAXITEMS ; j++ )
    {
        item_total[j] = 0;
        for( i = 0 ; i < MAXGIRLS ; i++ )
            item_total[j] = item_total[j] + value[i][j];
    }

    /* .....COMPUTING grand_total ..... */
    grand_total = 0;
    for( i = 0 ; i < MAXGIRLS ; i++ )
        grand_total = grand_total + girl_total[i];

    /* .....PRINTING OF RESULTS ..... */
    printf("\n GIRLS TOTALS\n\n");
    for( i = 0 ; i < MAXGIRLS ; i++ )
        printf("Salesgirl[%d] = %d\n", i+1, girl_total[i] );
    printf("\n ITEM TOTALS\n\n");
    for( j = 0 ; j < MAXITEMS ; j++ )
        printf("Item[%d] = %d\n", j+1, item_total[j] );
    printf("\nGrand Total = %d\n", grand_total);
}

```

Output

Input data

Enter values, one at a time, row_wise

```

310 257 365
210 190 325
405 235 240
260 300 380

```

GIRLS TOTALS

```

Salesgirl[1] = 950
Salesgirl[2] = 725
Salesgirl[3] = 880
Salesgirl[4] = 940

```

ITEM TOTALS

```

Item[1] = 1185
Item[2] = 1000
Item[3] = 1310

```

Grand Total = 3495

Fig. 7.4 Illustration of two-dimensional arrays

Example 7.4

Write a program to compute and print a multiplication table for numbers 1 to 5 as shown below:

	1	2	3	4	5
1	1	2	3	4	5
2	2	4	6	8	10
3	3	6	.	.	.
4	4	8	.	.	.
5	5	10	.	.	25

The program shown in Fig. 7.5 uses a two-dimensional array to store the table values. Each value is calculated using the control variables of the nested for loops as follows:

product[i][j] = row * column

where *i* denotes rows and *j* denotes columns of the **product** table. Since the indices *i* and *j* range from 0 to 4, we have introduced the following transformation:

row = *i*+1
column = *j*+1

7.4 INITIALIZING TWO-DIMENSIONAL ARRAYS

Like the one-dimensional arrays, two-dimensional arrays may be initialized by following their declaration with a list of initial values enclosed in braces. For example,

static int table[2][3] = { 0,0,0,1,1,1};

initializes the elements of the first row to zero and the second row to one. The initialization is done row by row. The above statement can be equivalently written as

static int table[2][3] = {{0,0,0}, {1,1,1}};

by surrounding the elements of each row by braces.

We can also initialize a two-dimensional array in the form of a matrix as shown below:

static int table[2][3] = {

{0,0,0},

{1,1,1}

};

Note the syntax of the above statements. Commas are required after each brace that closes off a row, except in the case of the last row.

If the values are missing in an initializer, they are automatically set to zero. For instance, the statement

Program

```

/*****
 *      PROGRAM TO PRINT MULTIPLICATION TABLE
 *****/

#define   ROWS      5
#define   COLUMNS  5

main( )
{
    int    row, column, product[ROWS][COLUMNS] ;
    int    i, j ;

    printf(" MULTIPLICATION TABLE\n\n");
    printf(" ");
    for( j = 1 ; j <= COLUMNS ; j++ )
        printf("%4d", j) ;
    printf("\n");
    printf("_____ \n");
    for( i = 0 ; i < ROWS ; i++ )
    {
        row = i + 1 ;
        printf("%2d |", row) ;
        for(j = 1 ; j <= COLUMNS ; j++ )
        {
            column = j ;
            product[i][j] = row * column ;
            printf("%4d", product[i][j] ) ;
        }
        printf("\n") ;
    }
}

```

Output

MULTIPLICATION TABLE					
	1	2	3	4	5
1	1	2	3	4	5
2	2	4	6	8	10
3	3	6	9	12	15
4	4	8	12	16	20
5	5	10	15	20	25

Fig. 7.5 Program to print multiplication table, using two-dimensional array

```

static int table[2][3] = {
                                {1,1},
                                {2}
};

```

will initialize the first two elements of the first row to one, the first element of the second row to two, and all other elements to zero.

When all the elements are to be initialized to zero, the following short-cut method may be used.

```
static int m[3][5] = { {0}, {0}, {0} };
```

The first element of each row is explicitly initialized to zero while other elements are automatically initialized to zero.

Example 7.5

A survey to know the popularity of four cars (Ambassador, Fiat, Dolphin and Maruti) was conducted in four cities (Bombay, Calcutta, Delhi and Madras). Each person surveyed was asked to give his city and the type of car he was using. The results, in coded form, are tabulated as follows:

```
M 1 C 2 B 1 D 3 M 2 B 4
C 1 D 3 M 4 B 2 D 1 C 3
D 4 D 4 M 1 M 1 B 3 B 3
C 1 C 1 C 2 M 4 M 4 C 2
D 1 C 2 B 3 M 1 B 1 C 2
D 3 M 4 C 1 D 2 M 3 B 4
```

Codes represent the following information:

M – Madras	1 – Ambassador
D – Delhi	2 – Fiat
C – Calcutta	3 – Dolphin
B – Bombay	4 – Maruti

Write a program to produce a table showing popularity of various cars in four cities.

A two-dimensional array **frequency** is used as an accumulator to store the number of cars used, under various categories in each city. For example, the element **frequency** [*i*][*j*] denotes the number of cars of type *j* used in city *i*. The **frequency** is declared as an array of size 5x5 and all the elements are initialized to zero.

The program shown in Fig. 7.6 reads the city code and the car code, one set after another, from the terminal. Tabulation ends when the letter X is read in place of a city code.

7.5 MULTIDIMENSIONAL ARRAYS

C allows arrays of three or more dimensions. The exact limit is determined by the compiler. The general form of a multidimensional array is

```
type array_name [s1][s2][s3]...[sm];
```

where *s_i* is the size of the *i*th dimension. Some example are:

```
int survey[3][5][12];
float table[5][4][5][3];
```

survey is a three-dimensional array declared to contain 180 integer type elements. Similarly **table** is a four-dimensional array containing 300 elements of floating-point type.

The array **survey** may represent a survey data of rainfall during the last three years from January to December in five cities.

```

Program
/*****
PROGRAM TO TABULATE SURVEY DATA
*****/

main( )
{
    int      i, j, car;
    static int frequency[5][5] = { {0}, {0}, {0}, {0}, {0} };
    char      city;
    printf("For each person, enter the city code \n");
    printf("followed by the car code.\n");
    printf("Enter the letter X to indicate end.\n");
    /* .....TABULATION BEGINS..... */
    for( i = 1 ; i < 100 ; i++ )
    {
        scanf("%c", &city );
        if( city == 'X' )
            break;
        scanf("%d", &car );
        switch(city)
        {
            case 'B' : frequency[1][car]++;
                        break;
            case 'C' : frequency[2][car]++;
                        break;
            case 'D' : frequency[3][car]++;
                        break;
            case 'M' : frequency[4][car]++;
                        break;
        }
    }
    /* .....TABULATION COMPLETED AND PRINTING BEGINS.. */
    printf("\n\n");
    printf("          POPULARITY TABLE\n\n");
    printf("-----\n");
    printf("City      Ambassador      Fiat      Dolphin      Maruti \n");
    printf("-----\n");
    for(i = 1; i <= 4; i++)
    {
        switch(i)
        {
            case 1: printf("Bombay      ");
                    break;
            case 2: printf("Calcutta    ");
                    break;
            case 3: printf("Delhi       ");
                    break;
            case 4: printf("Madras      ");
                    break;
        }
        for( j = 1 ; j <= 4 ; j++ )
            printf("%7d", frequency[i][j] );
    }
}

```

```
        printf("\n") ;
    }
    printf(" _____\n");
/* .....PRINTING ENDS..... */
}
```

Output

For each person, enter the city code followed by the car code.
Enter the letter X to indicate end.
M 1 C 2 B 1 D 3 M 2 B 4
C 1 D 3 M 4 B 2 D 1 C 3
D 4 D 4 M 1 M 1 B 3 B 3
C 1 C 1 C 2 M 4 M 4 C 2
D 1 C 2 B 3 M 1 B 1 C 2
D 3 M 4 C 1 D 2 M 3 B 4 X

POPULARITY TABLE

City	Ambassador	Fiat	Dolphin	Maruti
Bombay	2	1	3	2
Calcutta	4	5	1	0
Delhi	2	1	3	2
Madras	4	1	1	4

Fig. 7.6 Program to tabulate a survey data

If the first index denotes year, the second city and the third month, then the element `survey[1][2][9]` denotes the rainfall in the month of October during the second year in city-3.

Remember that a three-dimensional array can be represented as a series of two-dimensional arrays as shown below:

Year 1	month city	1	2		12
	1				
	. 5				
Year 2	month city	1	2		12
	1				
	. 5				

CASE STUDIES

1. Median of a List of Numbers

When all the items in a list are arranged in order, the middle value which divides the items into two parts with equal number of items on either side is called the *median*. Odd number of items have just one middle value while even number of items have two middle values. The median for even number of items is therefore designated as the average of the two middle values.

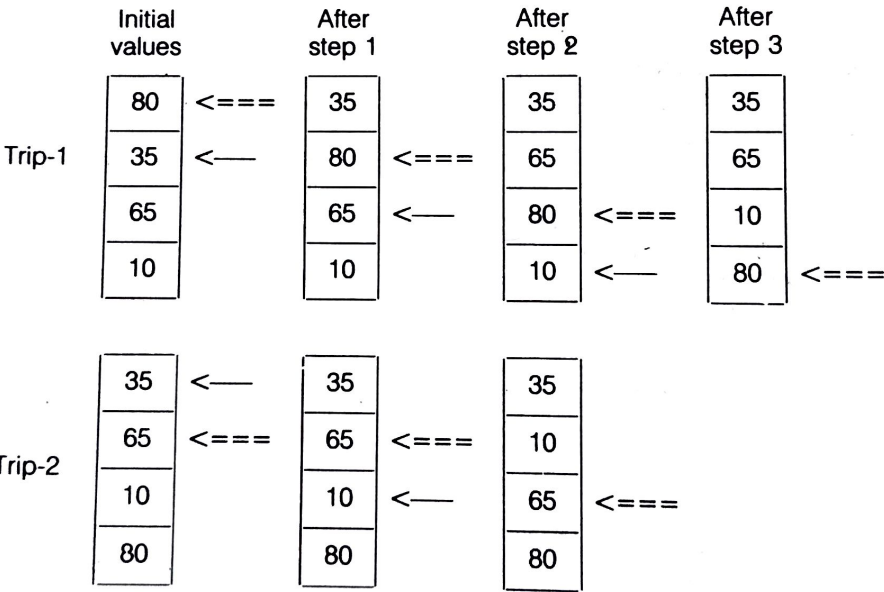
The major steps for finding the median are as follows:

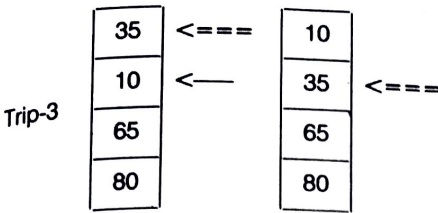
- 1. Read the items into an array while keeping a count of the items.
- 2. Sort the items in increasing order.
- 3. Compute median.

The program and sample output are shown in Fig. 7.7. The sorting algorithm used is as follows:

- 1. Compare the first two elements in the list, say $a[1]$, and $a[2]$. If $a[2]$ is smaller than $a[1]$, then interchange their values.
- 2. Compare $a[2]$ and $a[3]$; interchange them if $a[3]$ is smaller than $a[2]$.
- 3. Continue this process till the last two elements are compared and interchanged.
- 4. Repeat the above steps $n-1$ times.

In repeated trips through the array, the smallest elements 'bubble up' to the top. Because of this bubbling up effect, this algorithm is called *bubble sorting*. The bubbling effect is illustrated below for four items.





During the first trip, three pairs of items are compared and interchanged whenever needed. It should be noted that the number 80, the largest among the items, has been moved to the bottom at the end of the first trip. This means that the element 80 (the last item in the new list) need not be considered any further. Therefore, trip-2 requires only two pairs to be compared. This time, the number 65 (the second largest value) has been moved down the list. Notice that each trip brings the smallest value 10 up by one level.

The number of steps required in a trip is reduced by one for each trip made. The entire process will be over when a trip contains only one step. If the list contains n elements, then the number of comparisons involved would be $n(n-1)/2$.

Program

```

/*****
/*      PROGRAM TO SORT A LIST AND FIND ITS MEDIAN      */
*****/

#define N 10

main( )
{
    int i,j,h;
    float median,a[N], t;
    printf("Enter the number of items\n");
    scanf("%d", &n);

    /* Reading items into array a */
    printf("Input %d values \n",n);
    for (i = 1; i <= n ; i++)
        scanf("%f", &a[i]);

    /* Sorting begins */
    for (i = 1 ; i <= n-1 ; i++)
    { /* Trip-i begins */
        for (j = 1 ; j <= n-i ; j++)
        {
            if (a[j] <= a[j+1])
            { /* Interchanging values */
                t = a[j];
                a[j] = a[j+1];
                a[j+1] = t;
            }
        }
    }
    else

```

```

        continue ;
    } /* sorting ends */
/* calculation of median */
    if (n % 2 == 0)
        median = (a[n/2] + a[n/2 + 1])/2.0 ;
    else
        median = a[n/2 + 1];
/* Printing */
    for (i = 1 ; i <= n ; i++)
        printf("%f ", a[i]);
    printf("\n\nMedian is %f\n", median);
}

```

Output

Enter the number of items

5

Input 5 values

1.111 2.222 3.333 4.444 5.555
 5.555000 4.444000 3.333000 2.222000 1.111000

Median is 3.333000

Enter the number of items

6

Input 6 values

3 5 8 9 4 6
 9.000000 8.000000 6.000000 5.000000 4.000000 3.000000

Median is 5.500000

Fig. 7.7 Program to sort a list of numbers and to determine median

2. Calculation of Standard Deviation

In statistics, standard deviation is used to measure deviation of data from its mean. The formula for calculating standard deviation of n items is

$$s = \sqrt{\text{variance}}$$

where

$$\text{variance} = \frac{1}{n} \sum_{i=1}^n (x_i - m)^2$$

and

$$m = \text{mean} = \frac{1}{n} \sum_{i=1}^n x_i$$

The algorithm for calculating the standard deviation is as follows:

1. Read n items.
2. Calculate sum and mean of the items.
3. Calculate variance.
4. Calculate standard deviation.

Complete program with sample output is shown in Fig. 7.8.

Program

```

/*****
/*      PROGRAM TO CALCULATE STANDARD DEVIATION      */
*****/

#include <math.h>
#define MAXSIZE 100

main( )
{
    int i,n;
    float value [MAXSIZE], deviation,
          sum,sumsqr,mean,variance,stddeviation;

    sum = sumsqr = n = 0;
    printf("Input values: input -1 to end \n");
    for (i=1; i< MAXSIZE; i++)
    {
        scanf("%f", &value[i]);
        if (value[i] == -1)
            break;
        sum += value[i];
        n += 1;
    }
    mean = sum/(float)n;
    for (i = 1 ; i <= n; i++)
    {
        deviation = value[i] - mean;
        sumsqr += deviation * deviation;
    }
    variance = sumsqr/(float)n ;
    stddeviation = sqrt(variance) ;

    printf("\nNumber of items : %d\n",n);
    printf("Mean : %f\n", mean);
    printf("Standard deviation : %f\n", stddeviation);
}

```

Output

```

Input values: input -1 to end
65 9 27 78 12 20 33 49 -1

Number of items : 8
Mean : 36.625000
Standard deviation : 23.510303

```

Fig. 7.8 Program to calculate standard deviation

3. Evaluating a Test

A test consisting of 25 multiple-choice items is administered to a batch of 3 students. Correct answers and student responses are tabulated as shown below:

	Items																								
	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5
Correct answers																									
Student 1																									
Student 2																									
Student 3																									

Algorithm for evaluating the answers of students is as follows:

1. Read correct answers into an array.
2. Read the responses of a student and count the correct ones.
3. Repeat step-2 for each student.
4. Print the results.

A program to implement this algorithm is given in Fig. 7.9. The program uses the following arrays:

key[i] — To store correct answers of items
 response[i] — To store responses of students
 correct[i] — To identify items that are answered correctly.

```

Program
/*****
/* PROGRAM TO EVALUATE A MULTIPLE-CHOICE TEST */
*****/

#define STUDENTS 3
#define ITEMS 25
main()
{
    char key[ITEMS+1], response[ITEMS+1];
    int count, i, student, n,
        correct[ITEMS+1];

    /* Reading of Correct answers */
    printf("Input key to the items\n");
    for(i=0; i < ITEMS; i++)
        scanf("%c",&key[i]);
    scanf("%c",&key[i]);
    key[i] = '\0';

    /* Evaluation begins */

```

```

for(student = 1; student <= STUDENTS; student++)
{
    /*Reading student responses and counting correct ones*/
    count = 0;
    printf("\n");
    printf("Input responses of student-%d\n",student);
    for(i=0; i < ITEMS ; i++)
        scanf("%c",&response[i]);
    scanf("%c",&response[i]);
    response[i] = '\0';
    for (i = 0; i < ITEMS; i++)
        correct[i] = 0;
    for(i=0; i < ITEMS ; i++)
        if(response[i] == key[i])
        {
            count = count + 1;
            correct[i] = 1;
        }
    /* printing of results */
    printf("\n");
    printf("Student-%d\n", student);
    printf("Score is %d out of %d\n",count, ITEMS);
    printf("Response to the items below are wrong\n");
    n = 0;
    for(i=0; i < ITEMS ; i++)
        if(correct[i] == 0)
        {
            printf("%d",i+1);
            n = n+1;
        }
    if(n == 0)
        printf("NIL\n");
    printf("\n");
} /* Go to next student */

/* Evaluation and printing ends */
}

```

Output

Input key to the items
 abcdababcdababcdababcdab

Input responses of student-1
 abcdababcdababcdababcdab

Student-1
 Score is 25 out of 25
 Response to the following items are wrong
 NIL

Input responses of student-2
 abcdcdababcdabcccccccc

```
Student-2
Score is 14 out of 25
Response to the following items are wrong
5 6 7 8 17 18 19 21 22 23 25

Input responses of student-3
aaaaaaaaaaaaaaaaaaaaaaaaa

Student-3
Score is 7 out of 25
Response to the following items are wrong
2 3 4 6 7 8 10 11 12 14 15 16 18 19 20 22 23 24
```

Fig. 7.9 Program to evaluate responses to a multiple-choice test

4. Production and Sales Analysis

A company manufactures five categories of products and the number of items manufactured and sold are recorded product-wise every week in a month. The company reviews its production schedule at every month-end. The review may require one or more of the following information:

- (a) Value of weekly production and sales.
- (b) Total value of all the products manufactured.
- (c) Total value of all the products sold.
- (d) Total value of each product, manufactured and sold.

Let us represent the products manufactured and sold by two two-dimensional arrays M and S respectively. Then,

M =

M11	M12	M13	M14	M15
M21	M22	M23	M24	M25
M31	M32	M33	M34	M35
M41	M42	M43	M44	M45

S =

S11	S12	S13	S14	S15
S21	S22	S23	S24	S25
S31	S32	S33	S34	S35
S41	S42	S43	S44	S45

where M_{ij} represents the number of j th type product manufactured in i th week and S_{ij} the number of j th product sold in i th week. We may also represent the cost of each product by a single dimensional array C as follows:

C =

C1	C2	C3	C4	C5
----	----	----	----	----

where C_j is the cost of j th type product.

We shall represent the value of products manufactured and sold by two value arrays, namely, **Mvalue** and **Svalue**. Then,

$$\begin{aligned} \text{Mvalue}[i][j] &= M_{ij} \times C_j \\ \text{Svalue}[i][j] &= S_{ij} \times C_j \end{aligned}$$

A program to generate the required outputs for the review meeting is shown in Fig. 7.10. The following additional variables are used:

Mweek[i]	=	Value of all the products manufactured in week i.
	=	$\sum_{j=1}^5 \text{Mvalue}[i][j]$
Sweek[i]	=	Value of all the products sold in week i
	=	$\sum_{j=1}^5 \text{Svalue}[i][j]$
Mproduct[j]	=	Value of jth type product manufactured during the month
	=	$\sum_{i=1}^4 \text{Mvalue}[i][j]$
Sproduct[j]	=	Value of jth type product sold during the month
	=	$\sum_{i=1}^4 \text{Svalue}[i][j]$
Mtotal	=	Total value of all the products manufactured during the month
	=	$\sum_{i=1}^4 \text{Mweek}[i] = \sum_{j=1}^5 \text{Mproduct}[j]$
Stotal	=	Total value of all the products sold during the month
	=	$\sum_{i=1}^4 \text{Sweek}[i] = \sum_{j=1}^5 \text{Sproduct}[j]$

Program

```

/*****
/*          PRODUCTION AND SALES ANALYSIS          */
*****/

main( )
{
    int M[5][6], S[5][6], C[6],
        Mvalue[5][6], Svalue[5][6],
        Mweek[5], Sweek[5],
        Mproduct[6], Sproduct[6],
        Mtotal, Stotal, i, j, number;

    /*      Input data      */

    printf(" Enter products manufactured week_wise \n");
    printf(" M11,M12,——, M21,M22,—— etc\n");
    for(i=1; i<=4; i++)
        for(j=1; j<=5; j++)
            scanf("%d", &M[i][j]);

```

```

printf(" Enter products sold week_wise\n ");
printf(" S11,S12,———, S21,S22,——— etc\n ");
for(i=1; i<=4; i++)
    for(j=1; j<=5; j++)
        scanf("%d", &S[i][j]);
printf(" Enter cost of each product\n");
for(j=1; j <=5; j++)
    scanf("%d",&C[j]);
/* Value matrices of production and sales */
for(i=1; i<=4; i++)
    for(j=1; j<=5; j++)
    {
        Mvalue[i][j] = M[i][j] * C[j];
        Svalue[i][j] = S[i][j] * C[j];
    }
/* Total value of weekly production and sales */
for(i=1; i<=4; i++)
{
    Mweek[i] = 0 ;
    Sweek[i] = 0 ;
    for(j=1; j<=5; j++)
    {
        Mweek[i] += Mvalue[i][j];
        Sweek[i] += Svalue[i][j];
    }
}
/* Monthly value of product_wise production and sales */
for(j=1; j<=5; j++)
{
    Mproduct[j] = 0 ;
    Sproduct[j] = 0 ;
    for(i=1; i<=4; i++)
    {
        Mproduct[j] += Mvalue[i][j];
        Sproduct[j] += Svalue[i][j];
    }
}
/* Grand total of production and sales values */
Mtotal = Stotal = 0;
for(i=1; i<=4; i++)
{
    Mtotal += Mweek[i];
    Stotal += Sweek[i];
}
/ *****
                        Selection and printing of information required
*****/

printf("\n\n");
printf(" Following is the list of things you can\n");

```

```

printf(" request for. Enter appropriate item number\n");
printf(" and press RETURN Key\n\n");
printf(" 1.Value matrices of production & sales\n");
printf(" 2.Total value of weekly production & sales\n");
printf(" 3.Product_wise monthly value of production &");
printf(" sales\n");
printf(" 4.Grand total value of production & sales\n");
printf(" 5.Exit\n");

number = 0;
while(1)
{
    /* Beginning of while loop */
    printf("\n\n ENTER YOUR CHOICE:");
    scanf("%d",&number);
    printf("\n");
    if(number == 5)
    {
        printf("G O O D   B Y E\n\n");
        break;
    }
    switch(number)
    { /* Beginning of switch */
/*  VALUE MATRICES */
        case 1:
            printf("VALUE MATRIX OF PRODUCTION\n\n");
            for(i=1; i<=4; i++)
            {
                printf(" Week(%d)\t", i);
                for(j=1; j<=5; j++)
                    printf("%7d", Mvalue[i][j]);
                printf("\n");
            }
            printf("\n VALUE MATRIX OF SALES\n\n");
            for(i=1; i<=4; i++)
            {
                printf(" Week(%d)\t", i);
                for(j=1; j<=5; j++)
                    printf("%7d", Svalue[i][j]);
                printf("\n");
            }
            break;
/*  WEEKLY ANALYSIS */
        case 2:
            printf(" TOTAL WEEKLY PRODUCTION & SALES\n\n");
            printf("                PRODUCTION    SALES\n");
            printf("                _____    _____\n");
            for(i=1; i<=4; i++)
            {
                printf(" Week(%d)\t", i);

```

```

        printf("%7d\t%7d\n", Mweek[i], Sweek[i]);
    }
    break;
/*  PRODUCTWISE ANALYSIS  */
    case 3:
        printf(" PRODUCT_WISE TOTAL PRODUCTION &");
        printf(" SALES\n\n");
        printf("          PRODUCTION  SALES\n");
        printf("          _____  _____\n");
        for(j=1; j <=5; j++)
        {
            printf(" Product(%d)\t", j);
            printf("%7d\t%7d\n", Mproduct[j], Sproduct[j]);
        }
        break;
/*  GRAND TOTALS  */
    case 4:
        printf(" GRAND TOTAL OF PRODUCTION & SALES\n");
        printf("\n Total production = %d\n", Mtotal);
        printf(" Total sales      = %d\n", Stotal);
        break;
/*  DEFAULT  */
    default :
        printf(" Wrong choice, select again\n\n");
        break;
    } /* End of switch */
} /* End of while loop */
printf(" Exit from the program\n\n");
} /* End of main */

```

Output

Enter products manufactured week_wise
M11,M12,_____, M21,M22,_____ etc

```

11  15  12  14  13
13  13  14  15  12
12  16  10  15  14
14  11  15  13  12

```

Enter products sold week_wise
S11,S12,_____, S21,S22,_____ etc

```

10  13   9  12  11
12  10  12  14  10
11  14  10  14  12
12  10  13  11  10

```

Enter cost of each product
10 20 30 15 25

Following is the list of things you can request for. Enter appropriate item number and press RETURN key

1. Value matrices of production & sales
2. Total value of weekly production & sales
3. Product_wise monthly value of production & sales
4. Grand total value of production & sales
5. Exit

ENTER YOUR CHOICE:1

VALUE MATRIX OF PRODUCTION

Week(1)	110	300	360	210	325
Week(2)	130	260	420	225	300
Week(3)	120	320	300	225	350
Week(4)	140	220	450	185	300

VALUE MATRIX OF SALES

Week(1)	100	260	270	180	275
Week(2)	120	200	360	210	250
Week(3)	110	280	300	210	300
Week(4)	120	200	390	165	250

ENTER YOUR CHOICE 2

TOTAL WEEKLY PRODUCTION & SALES

	<u>PRODUCTION</u>	<u>SALES</u>
Week(1)	1305	1085
Week(2)	1335	1140
Week(3)	1315	1200
Week(4)	1305	1125

ENTER YOUR CHOICE:3

PRODUCT_WISE TOTAL PRODUCTION SALES

	<u>PRODUCTION</u>	<u>SALES</u>
Product(1)	500	450
Product(2)	1100	940
Product(3)	1530	1320
Product(4)	855	765
Product(5)	1275	1075

ENTER YOUR CHOICE:4

GRAND TOTAL OF PRODUCTION & SALES

Total production = 5260
Total sales = 4550

ENTER YOUR CHOICE:5

GOOD BYE

Exit from the program

Fig. 7.10 Program for production and sales analysis

REVIEW QUESTIONS AND EXERCISES

7.1 Explain the need for array variables.

7.2 Identify errors, if any, in each of the following array declaration statements.

- (a) `int score (100);`
- (b) `float values[10,15];`
- (c) `float average[ROW],[COLUMN];`
- (d) `char name[15];`
- (e) `int sum[ ];`

7.3 Identify errors, if any, in each of the following initialization statements.

- (a) `static int number[ ] = {0,0,0,0,0};`
- (b) `static float item[3][2] = {0,1,2,3,4,5};`
- (c) `static char word[ ] = {'A','R','R','A','Y'};`
- (d) `static int m[2,4] = {(0,0,0,0)(1,1,1,1)};`
- (e) `static float result[10] = 0;`

7.4 Assume that the arrays A and B are declared as follows:

```
int    A[5][4];
float  B[4];
```

Find the errors (if any) in the following program segments.

- (a) `for (i=1; i<=5; i++)`
`for(j=1; j<=4; j++)`
`A[i][j] = 0;`
- (b) `for (i=1; i<4; i++)`
`scanf("%f", B[i]);`
- (c) `for (i=0; i<=4; i++)`
`B[i] = B[i]+i;`
- (d) `for (i=4; i>=0; i--)`
`for (j=0; j<4; j++)`
`A[i][j] = B[j] + 1.0;`

7.5 Write the segment of a program that initializes the array A as follows:

1	0	0	0	0	...	0
0	1	0	0	0	...	0
0	0	1	0	0	...	0
.	.	.	.	.		.
.	.	.	.	.		.
.	.	.	.	.		.
.	.	.	.	.		.
0	0	0	0	0	...	1

All diagonal elements are initialized to one and others to zero.

7.6 Write a program for fitting a straight line through a set of points (x_i, y_i) , $i = 1, \dots, n$. The straight line equation is

$$y = mx + c$$

and the values of m and c are given by

$$m = \frac{n \sum (x_i y_i) - (\sum x_i)(\sum y_i)}{n (\sum x_i^2) - (\sum x_i)^2}$$

$$c = \frac{1}{n} (\sum y_i - m \sum x_i)$$

All summations are from 1 to n .

7.7 The daily maximum temperatures recorded in 10 cities during the month of January (for all 31 days) have been tabulated as follows:

	City									
	1	2	3	...	...	...	...	...	...	10
Day										
1										
2										
3										
.	.									
.	.									
.	.									
.	.									
31										

Write a program to read the table elements into a two-dimensional array **temperature**, and to find the city and day corresponding to (a) the highest temperature and (b) the lowest temperature.

7.8 An election is contested by 5 candidates. The candidates are numbered 1 to 5 and the voting is done by marking the candidate number on the ballot paper. Write a program to read the ballots and count the votes cast for each candidate using an array variable **count**. In case, a number read is outside the range 1 to 5, the ballot should be considered as a 'spoilt ballot' and the program should also count the number of spoilt ballots.

7.9 The following set of numbers is popularly known as Pascal's triangle.

1					
1	1				
1	2	1			
1	3	3	1		
1	4	6	4	1	
1	5	10	10	5	1
.	.	.	.	.	.
.	.	.	.	.	.

If we denote rows by i and columns by j , then any element (except the boundry elements) in the triangle is given by

$$p_{ij} = p_{i-1, j-1} + p_{i-1, j}$$

Write a program to calculate the elements of the Pascal triangle for 10 rows and print the results.

